Interpolation and Approximation Laboratory

Project Report - Scientific Computing, Project 9

Team: Mrutyunjay Malik (team leader), Rahul Kumar, Satrudhan Kumar, Vanlalthazama
Repository: github.com/Mrutyunjay-05/interpolation-approximation-lab

1. Introduction

The aim of this project is to build a small toolkit for approximating functions from sampled data, and to study how the number and placement of interpolation nodes affect the quality of the approximation. We implemented polynomial interpolation using Newton divided differences, without any black-box interpolation routine, and used it to run four experiments: a smooth function, an increasing number of nodes, a comparison of node placements, and the Runge function.

2. Method

2.1 Newton divided differences

Given distinct nodes x_0, ..., x_(n-1) and values y_0, ..., y_(n-1), the interpolating polynomial of degree at most n - 1 is written in Newton form:

```
p(x) = c_0 + c_1 (x - x_0) + c_2 (x - x_0)(x - x_1) + ...
+ c_(n-1) (x - x_0)...(x - x_(n-2))
```

The coefficients c_k are the divided differences f[x_0, ..., x_k], computed with the recurrence

```
f[x_i, ..., x_(i+k)] = ( f[x_(i+1), ..., x_(i+k)] - f[x_i, ..., x_(i+k-1)] )
/ ( x_(i+k) - x_i )
```

Computing all coefficients takes O(n^2) operations. The polynomial is then evaluated with a Horner-style nested scheme, which needs O(n) operations per query point. We chose the Newton form over the Lagrange form because the evaluation is simple and efficient, and because a new node only adds one more coefficient.

2.2 Input validation

All public calls go through one function, interpolate(x_nodes, y_nodes, xq). It raises a ValueError with a clear message when the inputs are not numeric, are not one-dimensional, are empty, have different lengths, contain NaN or infinity, or contain repeated nodes. The query points xq can be a scalar or an array of any shape, and the result has the same shape as xq.

2.3 Error measure and node choices

When the exact function f is known, the error is the maximum absolute error max |f(x) - p(x)| over a fine grid of 2001 equally spaced points on the interval. We compare two node distributions on an interval [a, b]:

- Equispaced nodes: x_k = a + k (b - a) / (n - 1), including both endpoints.
- Chebyshev nodes: x_k = (a + b)/2 + (b - a)/2 * cos((2k + 1) pi / (2n)), for k = 0, ..., n - 1. They cluster near the endpoints.

3. Interpolation and approximation

Interpolation constructs a function that passes exactly through the given data points. Approximation only requires the function to be close to the data or to the underlying function, for example a least-squares fit, and it need not pass through any point. An interpolating polynomial matches the data at the nodes, but between the nodes, and especially near the endpoints or outside the node interval, it can differ a lot from the underlying function.

For a sufficiently smooth f, the interpolation error at a point x is

```
f(x) - p(x) = f^(n)(xi) / n! * (x - x_0)(x - x_1)...(x - x_(n-1)),
```

for some xi in the interval spanned by the nodes and x. The product of the factors (x - x_i) is what the node placement controls, and it grows very quickly outside the node interval.

4. Experiments

All experiments are run with one command, python experiments/run_experiments.py, which saves the figures and tables in the figures folder.

4.1 Experiment A: a smooth function

We interpolate f(x) = sin(x) on [0, 2 pi] with n = 4, 6 and 8 equispaced nodes.

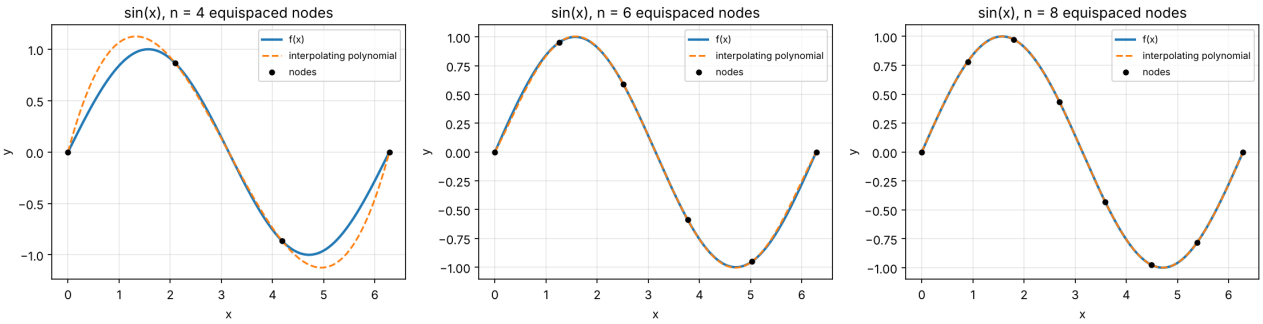


Figure 1. sin(x) and its interpolating polynomial for n = 4, 6, 8 equispaced nodes.

n	maximum absolute error
4	2.55e-01
6	2.68e-02
8	1.70e-03

The polynomial is already very close to sin(x) between the nodes at n = 8, and the error drops by roughly a factor of ten for every two extra nodes. For a smooth function, low-degree interpolation is accurate.

4.2 Experiment B: effect of the number of nodes

We repeat the experiment for sin(x) on [0, 2 pi] with n = 3, 6, ..., 30 nodes and record the maximum error for both node types.

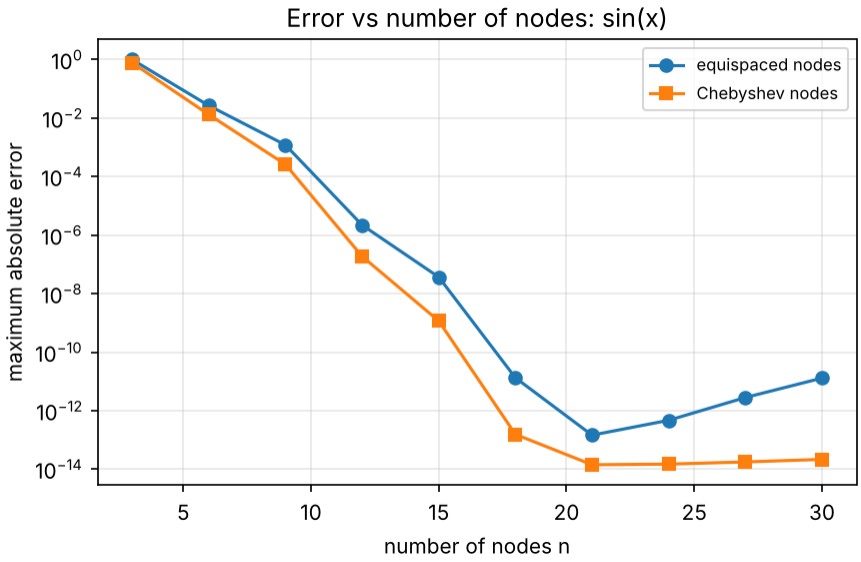


Figure 2. Maximum error against the number of nodes for sin(x) (log scale).

n	equispaced nodes	Chebyshev nodes
3	1.0e+00	7.8e-01
6	2.7e-02	1.3e-02
9	1.2e-03	2.6e-04
12	2.2e-06	1.9e-07
15	3.7e-08	1.2e-09
18	1.3e-11	1.6e-13
21	1.5e-13	1.4e-14
24	4.7e-13	1.5e-14
27	2.8e-12	1.8e-14
30	1.3e-11	2.2e-14

Adding nodes improves the result consistently only up to about n = 21. Beyond that, the error with equispaced nodes stops decreasing and slowly grows again, from 1.5e-13 at n = 21 to 1.3e-11 at n = 30. This is a numerical effect: the high-degree polynomial is ill-conditioned for equispaced nodes, so rounding errors are amplified. With Chebyshev nodes the error stays near 1e-14. So more nodes do not always help, and the placement of the nodes matters.

4.3 Experiment C: node placement

We compare 11 equispaced nodes with 11 Chebyshev nodes for f(x) = 1 / (1 + x^2) on [-5, 5].

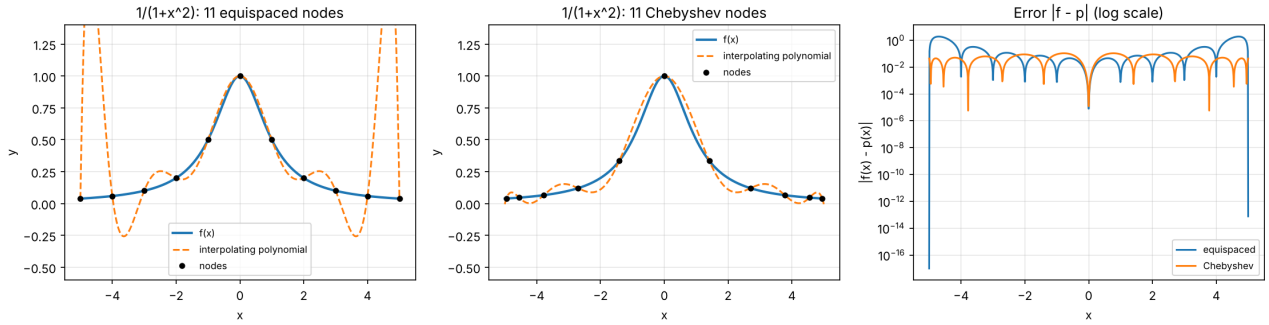


Figure 3. Equispaced nodes (left), Chebyshev nodes (middle) and the error |f - p| (right).

nodes	maximum absolute error
11 equispaced	1.92
11 Chebyshev	0.109

With equispaced nodes, the polynomial oscillates strongly near the endpoints. With Chebyshev nodes, which are denser near the endpoints, the oscillation is much smaller and the maximum error is about 18 times lower. The error curve in the third panel shows that the largest errors are concentrated near the ends of the interval.

4.4 Experiment D: Runge-type behaviour

We use the Runge function f(x) = 1 / (1 + 25 x^2) on [-1, 1] with n = 5, 10, 15 and 20 nodes. Note that the Runge function is only a test function: we interpolate it, we do not solve any equation with it.

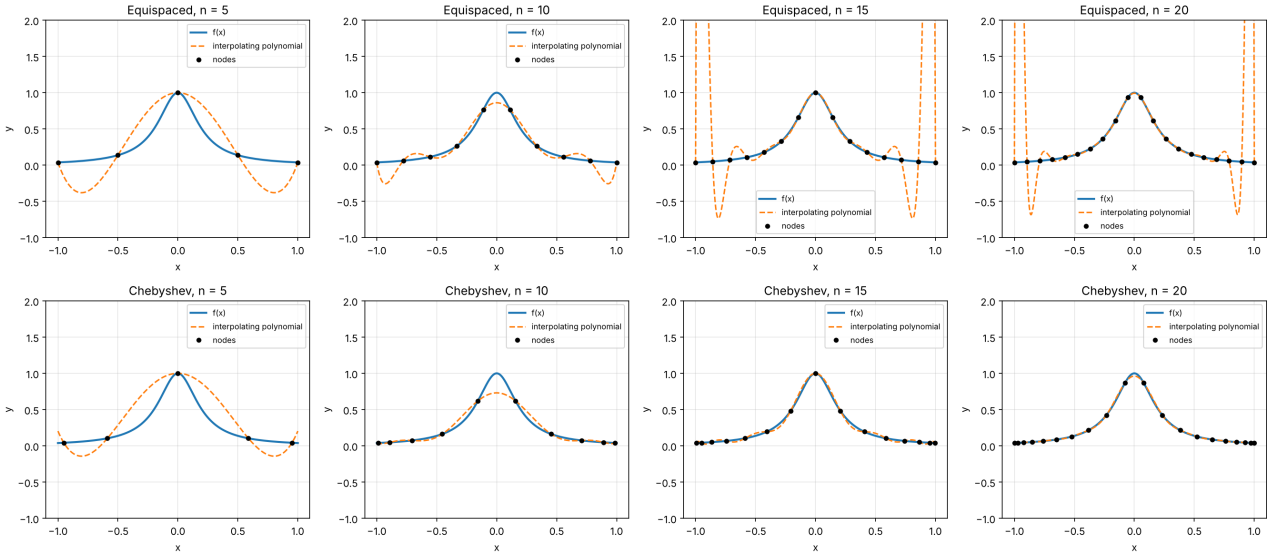


Figure 4. Runge function: equispaced nodes (top row) and Chebyshev nodes (bottom row).

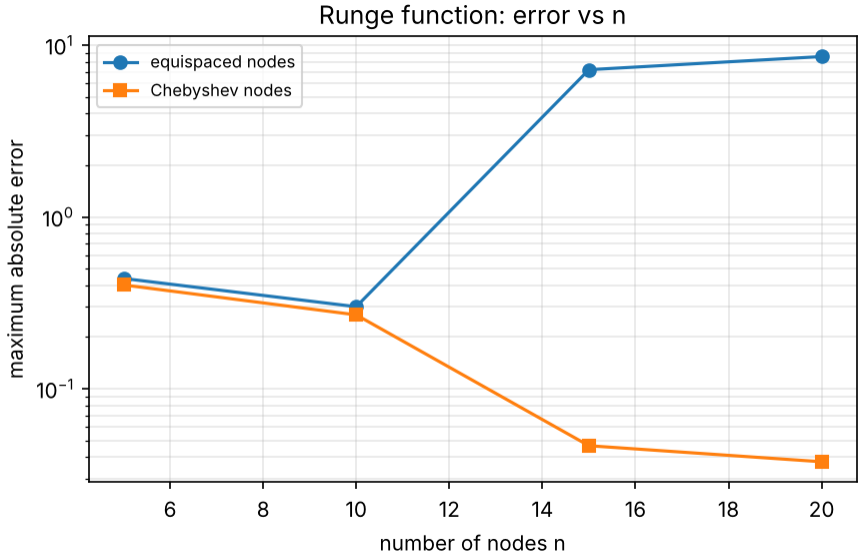


Figure 5. Maximum error against n for the Runge function.

n	equispaced nodes	Chebyshev nodes
5	0.44	0.40
10	0.30	0.27
15	7.2	0.047
20	8.6	0.038

For equispaced nodes, the error does not decrease when n grows: it goes from 0.30 at n = 10 to 7.2 at n = 15 and 8.6 at n = 20. The polynomial matches the function well near the centre but oscillates with growing amplitude close to x = -1 and x = 1. For Chebyshev nodes, the error keeps decreasing (0.27, 0.047, 0.038) and the oscillations near the endpoints disappear.

5. Interpretation: oscillations and endpoint behaviour

- Within the node range, interpolation of a smooth function is accurate (Experiments A and B), but accuracy depends on the node placement.
- Near the endpoints, equispaced nodes give an interpolation error whose node polynomial (x - x_0)...(x - x_(n-1)) is much larger than in the middle of the interval. For functions with a large derivative or poles close to the real axis, such as the Runge function, this makes the error grow with n (Runge phenomenon).
- Chebyshev nodes cluster near the endpoints and make the maximum of that product small and almost uniform over the interval. This is why the oscillations near the endpoints are suppressed in Experiments C and D.
- Outside the interval spanned by the nodes (extrapolation), the factors (x - x_i) grow quickly with the distance from the interval, so a polynomial interpolant cannot be trusted there. It is best used for interpolation inside the data range.
- In floating-point arithmetic, very high degrees with equispaced nodes also lead to rounding errors (Experiment B), which gives a second reason to avoid them.

6. Conclusion

Newton divided differences give a simple, validated and efficient implementation of polynomial interpolation. For smooth functions a small number of nodes is enough, but increasing the number of equispaced nodes does not guarantee a better approximation: the Runge function shows growing oscillations near the endpoints, and very high degrees amplify rounding errors. Chebyshev nodes fix both problems in our experiments, so they are the better default when the nodes can be chosen freely. If the data are given at equally spaced nodes, a low-degree interpolant or an approximation method such as least squares or piecewise polynomials is safer.

7. Team work and collaboration

The work was organised on GitHub with one issue per task, one feature branch per task, and pull requests into a protected main branch. A pull request could only be merged after another member approved it.

Member	Contribution	Main files
Mrutyunjay Malik (team leader)	Interpolation core, repository setup, integration, README, report	interp/core.py
Rahul Kumar	Evaluation and input validation	interp/evaluate.py
Satrudhan Kumar	Error analysis and node generators	interp/errors.py
Vanlalthazama	Experiments and visualization	interp/plotting.py, experiments/run_experiments.py

The repository contains 29 automated tests (python -m pytest), and the experiments can be reproduced on a clean clone with python experiments/run_experiments.py.